HiNotes Second Brain: "Ask AI" Feature — Product Design Document

Product: HiNotes (Companion App to HiDock P1)

Feature: Second Brain — LLM Knowledge Base for Meeting Intelligence

Version: 3.0

Date: April 5, 2026

Prepared by: Manus AI

Reviewed by: Staff Engineer, AI Infrastructure

---

1. Executive Summary

HiDock P1 records and transcribes every meeting. Today, those transcripts are a pile of raw material — searchable, but fundamentally inert. Each meeting is an island. This document designs a feature that changes that entirely.

The feature is called **HiNotes Second Brain**. It is powered by the **LLM Knowledge Base** architecture described by Andrej Karpathy [3]. Instead of answering questions by searching raw transcripts at query time (the RAG approach used by Plaud and Otter), HiNotes will deploy an LLM as an autonomous librarian. After each day's meetings, the LLM reads every new transcript and compiles it into a persistent, interlinked, human-readable Markdown wiki. People get profiles. Projects get trackers. Topics get wiki articles. Decisions get logged. Contradictions get flagged.

A key innovation is the **Three-Temperature Memory Architecture**: knowledge is organized into **Hot** (last 1 month), **Warm** (1–6 months), and **Cold** (6+ months) tiers. All tiers are stored as **plain-text Markdown** — Cold memory uses filesystem-level transparent compression (ZFS/btrfs) instead of application-level zip archives, so the LLM can always read any file directly without decompression overhead. Every tier maintains a well-designed index. A **hybrid search layer** (BM25 + lightweight vector search) enables fast parallel retrieval across all tiers simultaneously, while the LLM handles synthesis and reasoning.

Raw transcripts are **immutable and permanent** — they are never deleted, only moved to archival directories. The wiki is a derived artifact; the transcripts are the ground truth. The entire wiki is **Git-backed**, with every compilation cycle as an atomic commit, enabling instant rollback of any hallucinated update.

The result is a knowledge base that compounds over time. Every meeting makes it richer. The user's professional life — all the context, all the relationships, all the evolving strategies — becomes a living, navigable second brain with perfect long-term memory.

---

2. The Problem: Why RAG Fails Meetings

2.1 The Standard Approach and Its Ceiling

Dominant AI tools like Plaud and Otter use **Retrieval-Augmented Generation (RAG)**. They split documents into chunks, store them as vectors, and perform similarity searches at query time [3]. This works for "What did we discuss last Tuesday?" but fails for longitudinal, synthetic questions:

- "What is the current status of Project Nova, and how has it evolved since January?"
- "What has Sarah committed to across all her meetings this quarter?"
- "Is there a contradiction between what engineering said last week and what sales promised today?"

RAG fundamentally treats each query as a fresh start, rediscovering knowledge from scratch. As Karpathy observes, "Nothing is built up" [3].

2.2 The Specific Failure Mode for Meetings

Meeting transcripts are repetitive and conversational. A statement like "we'll go with the original plan" is meaningless without the context established weeks prior. RAG retrieves the statement but loses the link. Furthermore, the most valuable meeting intelligence is **relational** (who promised what to whom), which exists *across* meetings, not within individual chunks.

2.3 The Memory Problem: Everything or Nothing

Existing tools treat all meetings equally — either everything is live and searchable (expensive, slow) or old meetings are simply forgotten. Human memory doesn't work this way. We remember last week's meetings vividly, last quarter's meetings as summaries, and last year's meetings as key facts. HiNotes Second Brain mirrors this with a tiered memory system.

---

3. Three-Temperature Memory Architecture

Memory is organized into three tiers based on recency, each with its own content granularity, compilation frequency, and index. **All tiers are plain-text Markdown on disk** — no application-level compression. Storage efficiency is handled by filesystem-level transparent compression (ZFS/btrfs), which is invisible to the LLM and the search layer.

3.1 Overview

Property	🔴 Hot	🟡 Warm	🔵 Cold
Time Range	Last 1 month	1–6 months ago	6+ months ago
Storage Format	Markdown (plain text)	Markdown (plain text)	Markdown (plain text, fs-compressed)
Index	`hot/index.md`	`warm/index.md`	`cold/index.md`
Raw Transcripts	Full verbatim	Archived (read-only)	Archived (read-only)
Wiki Content	Full wiki pages	Full wiki pages	Condensed wiki pages
Search	Hybrid (BM25 + vector + index)	Hybrid (BM25 + vector + index)	Hybrid (BM25 + vector + index)
LLM Context Priority	Loaded first	Loaded on relevance	Loaded only on search hit
Compilation Frequency	After every meeting	Monthly re-synthesis	Quarterly condensation

3.2 Hot Memory (🔴 Last 1 Month)

Hot memory is the user's active working context. Everything is live Markdown, instantly accessible, and prioritized in the LLM's context window.

Directory Structure:

Plain Text

```
hot/
├── index.md              # Master index of all Hot content
├── people/               # Active person profiles
│   ├── sarah-chen.md
│   ├── marcus-lee.md
│   └── ...
├── projects/             # Active project trackers
│   ├── project-nova.md
│   ├── pricing-v2.md
│   └── ...
├── topics/               # Active topic articles
│   ├── pricing-strategy.md
│   └── ...
├── decisions/            # Recent decisions log
```

```
│   └── decisions-2026-04.md
├── actions/              # Active action items
│   └── actions-open.md
└── queries/              # Filed query answers
    └── ...
```

hot/index.md Design (optimized for LLM scanning — dense keywords, hierarchical bullets, minimal table overhead):

Markdown

```
# Hot Memory Index
> Updated: 2026-04-05 09:15 | 31 days | 47 meetings

## Recent Meetings
- 04-05 Morning Standup [Sarah, Marcus, Li] — Sprint 12 blockers, QA timeline d
- 04-04 Board Review [CEO, CFO, Sarah] — Q1 results, H2 budget approval $2.1M, 
- 04-03 Product Sync [Marcus, Design, Eng] — Pricing v2 legal review pending, l
- ...

## Active People (12)
- Sarah Chen (VP Eng) — 23 meetings, 3 open actions, Nova lead, key voice on hi
- Marcus Lee (Product Lead) — 18 meetings, 5 open actions (2 OVERDUE), pricing 
- ...

## Active Projects (4)
- Project Nova [🟢 On Track] — 78% complete, beta Apr 15, Sarah owns, last: sta
- Pricing V2 [🟡 At Risk] — pending legal review, decision Apr 8, Marcus owns,
- ...

## Open Actions (11, 3 overdue)
- ⚠️ Marcus: Pricing analysis v2 [due Apr 3, OVERDUE 2d] — source: Apr 1 standu
- Sarah: Announcement draft [due Apr 5, today] — source: Apr 4 board
- ...

## Recent Decisions (last 30d)
- Apr 4: Approved $2.1M H2 budget [Board Review, CFO]
- Apr 4: Deprioritized European expansion to 2027 [Board Review, CEO]
- Apr 3: Delay QA cycle to Monday [Standup, Sarah]
- ...
```

3.3 Warm Memory (🟡 1–6 Months)

Warm memory retains full wiki pages. Raw transcripts are moved to a read-only archive directory (never deleted). Wiki pages are re-synthesized monthly to consolidate overlapping information.

Directory Structure:

Plain Text

```
warm/
├── index.md              # Master index of all Warm content
├── people/               # Person profiles (historical context)
│   ├── sarah-chen.md
│   └── ...
├── projects/             # Project history (full evolution)
│   ├── project-nova.md
│   └── ...
├── topics/               # Topic articles
│   └── ...
├── decisions/            # Decision archive by month
│   ├── decisions-2026-03.md
│   └── ...
└── actions/              # Completed/expired action items
    └── actions-closed-q1.md
```

`warm/index.md` **Design:**

Markdown

```
# Warm Memory Index
> Updated: 2026-04-01 | 5 months | 203 meetings

## Monthly Summaries
- Mar 2026 (45 meetings) — Sprint 10-11, Q1 close, hired 3 engineers, froze Q2
- Feb 2026 (42 meetings) — Product roadmap finalized, board prep, approved Nova
- Jan 2026 (38 meetings) — Annual planning, set 2026 OKRs, $8M revenue target
- Dec 2025 (40 meetings) — Year-end review, Nova alpha shipped, holiday party p
- Nov 2025 (38 meetings) — Nova kickoff, Series A close-out, new office setup

## People (cumulative, 34 unique)
- Sarah Chen (VP Eng) — 89 meetings total, core architect, Nova lead since Nov
- Marcus Lee (Product Lead) — 72 meetings, pricing strategy owner since Jan 202
- James Wu (CTO, departed Jun 2025) — 45 meetings, set original tech stack, las
- ...

## Projects (full history)
- Project Nova — started Nov 2025, spec Dec, alpha Feb, beta target Apr, Sarah
- Pricing V2 — started Jan 2026, 3 iterations, currently blocked on legal
- Series A Close-out — completed Nov 2025, $5M raised, led by CEO
- ...

## Key Decisions (6 months)
```

```
- Mar 15: Hire 3 engineers [✅ all onboarded]
- Feb 20: Nova beta by April [🟢 on track]
- Jan 10: Set $8M 2026 revenue target
- Dec 5: Chose Stripe for payments [replaced PayPal]
- ...
```

3.4 Cold Memory (🔵 6+ Months)

Cold memory stores condensed wiki pages as **plain-text Markdown** — no zip, no archives. Filesystem-level transparent compression (ZFS/btrfs) handles storage efficiency invisibly. The LLM and search layer can read any Cold file directly, with zero decompression overhead.

Raw transcripts from this period are stored in a separate read-only archive directory, accessible but not indexed for routine queries.

Directory Structure:

Plain Text

```
cold/
├── index.md              # Master index of all Cold content
├── 2025-Q3/
│   ├── summary.md        # Quarter-level narrative summary
│   ├── people/           # Condensed person snapshots
│   ├── projects/         # Project outcome summaries
│   ├── topics/           # Condensed topic articles
│   └── decisions/        # Decision log with outcomes
├── 2025-Q2/
│   └── ...
├── 2025-Q1/
│   └── ...
└── ...

raw-archive/              # IMMUTABLE — never deleted, never modified
├── 2026-03/              # Demoted from Hot after 30 days
│   ├── 2026-03-05_standup.md
│   └── ...
├── 2025-Q3/              # Full verbatim transcripts
│   └── ...
└── ...
```

`cold/index.md` Design:

Markdown

```
# Cold Memory Index
> Updated: 2026-04-01 | All history before Oct 2025 | 312 meetings | 89 partici
```

```
## Quarterly Summaries
- 2025 Q3 (98 meetings) — Product launch v1 Aug 1, Series A closed $5M, team gr
  - Key: v1 launch strategy, investor relations, first 500 users acquired
- 2025 Q2 (87 meetings) — MVP development, hiring wave 1, chose tech stack (Rus
  - Key: architecture decisions, first 5 engineers hired, pivoted B2C→B2B
- 2025 Q1 (72 meetings) — Company founding, initial product concept, incorporat
  - Key: founding team formation, market research, seed funding

## People (all-time, 89 unique)
- James Wu (CTO, departed Jun 2025) — 45 meetings, set Rust/PostgreSQL stack, k
- Lisa Park (Designer, departed Aug 2025) — 30 meetings, designed original UI,
- ...

## Projects (completed/archived)
- V1 Launch — Q2-Q3 2025, shipped Aug 1, 500 users, Sarah+James led
- Series A — Q3 2025, raised $5M from Sequoia, CEO led
- B2C→B2B Pivot — Q2 2025, fundamental strategy shift, entire team involved
- ...

## Foundational Decisions (still relevant today)
- Aug 2025: Chose Rust for backend [still core stack today]
- Jun 2025: Pivoted B2C→B2B [defined current strategy]
- May 2025: First discussed Kubernetes [deferred, revisited Q1 2026]
- Apr 2025: PostgreSQL over MongoDB [still primary DB]
- ...
```

---

4. The Tiered Index System & Hybrid Search

4.1 The Master Index (`master-index.md`)

A single root-level file that provides a bird's-eye view across all three tiers:

Markdown

```
# HiNotes Second Brain — Master Index
> User: Sean | Total meetings: 562 | Span: Jan 2025 – Apr 2026
> Hot: 47 meetings (31 days) | Warm: 203 meetings (5 months) | Cold: 312 meetin

## Quick Stats
- Active people: 12 (Hot), 34 (Warm), 89 (All-time)
- Active projects: 4 (Hot), 8 (Warm), 15 (All-time)
- Open action items: 11 (3 overdue)
- Unresolved contradictions: 2
- Last compilation: 2026-04-05 02:15 UTC [commit a3f7b2c]
```

```
## Tier Pointers
- 🔴 Hot: → hot/index.md (last 30 days, full detail)
- 🟡 Warm: → warm/index.md (1-6 months, summaries)
- 🔵 Cold: → cold/index.md (6+ months, condensed)
- 📦 Raw Archive: → raw-archive/ (all transcripts, immutable)
```

4.2 Hybrid Search Layer

The LLM does **not** scan indexes sequentially. Instead, a **programmatic search layer** runs in parallel across all tiers before the LLM is invoked:

Component	Technology	Purpose
BM25 Full-Text	SQLite FTS5	Keyword matching across all Markdown files in all tiers
Vector Search	Lightweight embeddings (e.g., `all-MiniLM-L6-v2`)	Semantic similarity for fuzzy/conceptual queries
Index Scan	Deterministic string match	Fast lookup in index files for known entities (people, projects)

Query Resolution Flow:

Plain Text

```
User asks: "When did we first discuss moving to Kubernetes?"

Step 1: PARALLEL SEARCH (< 500ms)
  - BM25 hits: cold/2025-Q2/topics/tech-stack-decisions.md (score: 0.92)
  - BM25 hits: warm/topics/infrastructure.md (score: 0.45)
  - Vector hit: cold/2025-Q2/summary.md (score: 0.78)
  - Index scan: cold/index.md → "May 2025: First discussed Kubernetes"

Step 2: LLM SYNTHESIS
  - Context injected: cold/index.md snippet + cold/2025-Q2/topics/tech-stack-de
  - LLM reads the specific pages and synthesizes answer
  - Answer: "First discussed May 12, 2025 in the Architecture Review.
     James Wu proposed it. Decision was deferred to Q3. Revisited Jan 2026."

Step 3: OPTIONAL — cite raw source
  - If user wants proof: system retrieves raw-archive/2025-Q2/2025-05-12_arch-r
  - LLM highlights the exact passage with timestamp
```

This parallel approach eliminates the sequential tier-scanning bottleneck. Ambiguous queries that span multiple timeframes (e.g., "How has Sarah's role evolved?") retrieve candidates from all tiers simultaneously.

---

5. The Compilation Pipeline

5.1 Git-Backed Atomic Commits

The entire wiki directory is a **Git repository**. Every compilation cycle is a single atomic commit. This provides:

- **Version history:** Every state of the wiki is preserved.
- **Instant rollback:** If a compilation hallucinates, `git revert` restores the previous state.
- **Auditability:** Every change is traceable to the meeting that triggered it.
- **Diff visibility:** Users can see exactly what changed after each meeting.

5.2 Pipeline Steps (After Every Meeting)

1. **Entity Extraction:** LLM identifies people, projects, topics, decisions, and action items from the raw transcript.
2. **Draft Wiki Updates:** LLM drafts updates to 8–15 relevant wiki pages. Drafts are written to a staging area, not directly to the wiki.
3. **Deterministic Cross-Linking:** A script (not the LLM) scans drafted pages for known entity names and inserts Markdown links. This prevents hallucinated connections.
4. **Schema Validation:** Each drafted page is validated against `SCHEMA.md` templates. Malformed pages are rejected.
5. **Confidence Scoring:** Each update is tagged with a confidence score. Low-confidence updates (e.g., ambiguous speaker attribution) are flagged for user review.
6. **Atomic Commit:** All validated updates are committed to Git as a single transaction. Failed validations block the entire commit.
7. **Index Regeneration:** `hot/index.md` and `master-index.md` are regenerated deterministically from the wiki content (not by the LLM).
8. **Search Index Update:** BM25 and vector indexes are incrementally updated.
9. **Notification:** A summary of changes is queued for the user's **Morning Brief**.

5.3 Human-in-the-Loop: The Review Queue

Following Karpathy's principle that the user is the "editor-in-chief" [3]:

- **High-confidence updates** (>0.9): Auto-committed. User sees a summary in the Morning Brief.
- **Medium-confidence updates** (0.6–0.9): Auto-committed but flagged. User sees "⚠️ Review suggested" tags in the Morning Brief.
- **Low-confidence updates** (<0.6): Held in staging. User must approve or reject before they enter the wiki. Example: "I'm not sure if 'the original plan' refers to the March pricing or the February timeline. Please clarify."

5.4 Monthly Cycle (Hot → Warm Demotion)

On the 1st of each month:

1. **Archive Raw Transcripts:** Raw transcripts older than 30 days are moved to `raw-archive/` (never deleted).
2. **Re-synthesize:** Wiki pages that reference only old meetings are re-synthesized into more compact Warm-tier pages.
3. **Move:** Re-synthesized content moves from `hot/` to `warm/`.
4. **Re-index:** All indexes are regenerated. Search indexes are rebuilt.
5. **Git commit:** The entire demotion is a single atomic commit.

5.5 Quarterly Cycle (Warm → Cold Condensation)

Every quarter:

1. **Condense:** Warm wiki pages older than 6 months are further condensed — only key facts, decisions, and outcomes survive.
2. **Move:** Condensed content moves to `cold/{quarter}/`.
3. **Raw transcripts** from this period remain in `raw-archive/`, untouched.
4. **Re-index:** `cold/index.md` is updated. All search indexes are rebuilt.
5. **Lint:** The Lint Engine runs a full consistency check across all three tiers, flagging orphaned references, contradictions, and stale action items.
6. **Git commit:** Single atomic commit.

---

6. Raw Transcript Immutability

> **Design Principle:** Raw transcripts are the ground truth. The wiki is a derived artifact. If the wiki contradicts a transcript, the transcript is correct.

All raw transcripts are stored permanently in `raw-archive/`, organized by date. They are:

- **Never modified** by the LLM or any automated process.
- **Never deleted**, regardless of age.
- **Always citable** — every wiki statement includes a source reference like `[src: 2026-04-04_board-review.md @14:23]`.
- **Accessible on demand** — when a user questions a wiki claim, the system retrieves the exact passage from the raw transcript with timestamp.

Storage impact: A typical 1-hour meeting transcript is ~50KB of text. One year of daily meetings (250 days × 3 meetings) = ~37.5MB. With filesystem-level compression, this drops to ~10MB. Negligible.

---

7. Privacy and PII Handling

Meeting transcripts contain highly sensitive data. The system must address:

- **PII Detection:** Before compilation, a PII scanner identifies and tags sensitive information (SSNs, credit card numbers, salary figures, medical information).
- **PII Redaction in Wiki:** The wiki stores redacted versions. Raw transcripts retain full content but are access-controlled.
- **Local-First Processing:** On HiDock H2 (edge device), all compilation runs locally. No transcript data leaves the device unless the user explicitly opts into cloud processing.
- **Cloud Processing:** For cloud-hosted users, transcripts are encrypted at rest and in transit. Processing happens in ephemeral containers that are destroyed after compilation.
- **Data Retention Policies:** Users can set retention policies per tier. Cold memory can be configured to auto-purge after N years, but raw transcripts are retained separately per legal requirements.

---

8. The "Wow Moments" for Users

1. **The Living Profile:** Tapping a participant's name shows their full history assembled from Hot, Warm, and Cold memory seamlessly — role, meetings, commitments, positions on key topics, and how their stance has evolved over time.
2. **The Contradiction Alert:** A proactive alert: "⚠️ Conflict: In March you set the launch for Aug 15, but yesterday the team referenced July. Which is current?" User taps to see both source transcripts side by side.

3. **The Morning Brief:** A daily digest: "Yesterday's sync confirmed the budget. Marcus is 3 days overdue on the pricing analysis. Sarah committed to the announcement draft today. 2 wiki updates need your review."
4. **The Project Timeline:** Reconstruct the entire history of an initiative from its first mention (Cold) through its evolution (Warm) to its current status (Hot), with all rationale preserved and every claim linked to a source transcript.
5. **The Deep Memory:** "When did we first talk about Kubernetes?" — hybrid search finds it in Cold in <500ms, LLM reads the specific page, answers with the exact date, who proposed it, and what happened next. Users feel like the system remembers *everything*.
6. **The Undo Button:** "That last compilation got Sarah's role wrong." User taps revert, Git rolls back the commit, wiki is restored instantly.

---

9. Competitive Analysis

Feature	HiNotes Second Brain	Plaud Ask AI	Otter.ai
Architecture	LLM Knowledge Base + Hybrid Search	RAG (Vector)	RAG (Vector)
Knowledge Accumulation	**Yes (Compounding)**	No (Stateless)	No (Stateless)
Memory Tiers	**Hot/Warm/Cold**	None (flat)	None (flat)
Cross-Meeting Synthesis	High (Pre-compiled wiki)	Low (Chunk-based)	Moderate
Contradiction Detection	**Yes (Proactive)**	No	No
Long-term Memory	**Unlimited (all tiers searchable)**	Limited	Limited
Source Auditability	**Every claim cites raw transcript**	Low (Black box)	Low
Rollback	**Git-backed instant revert**	None	None
Privacy	**Local-first on H2, PII redaction**	Cloud-only	Cloud-only
Search	Hybrid (BM25 + vector + index)	Vector only	Vector only

10. Phased Rollout Plan

- **Phase 1 (Months 1-2):** Foundation. Git-backed wiki, Hot tier compilation pipeline, BM25 search, basic Ask AI over single meetings. Deterministic index generation. Confidence scoring with review queue.
- **Phase 2 (Months 3-4):** Knowledge Base. Launch "Knowledge Base" tab with People Profiles, Project Trackers, cross-meeting Ask AI using hybrid search. Morning Brief v1.
- **Phase 3 (Months 5-6):** Memory Tiers. Implement Warm demotion cycle, Cold condensation. Lint Engine for contradiction detection. PII scanner. Full three-tier hybrid search.
- **Phase 4 (Months 7-9):** Deep Memory & Polish. Visual knowledge graph, on-demand raw transcript citation, team-shared knowledge bases, advanced Morning Brief with trend analysis.

11. References

[1] Plaud.ai Support. "Ask Across All Files."

[2] Otter.ai. "Otter Meeting Agent — AI Notetaker, Transcription, Insights."

[3] Karpathy, A. (2026, April 2). "LLM Wiki — A pattern for building personal knowledge bases using LLMs." GitHub Gist.

[4] VentureBeat. (2026, April 3). "Karpathy shares 'LLM Knowledge Base' architecture that bypasses RAG."

[5] Lütke, T. (2026). "qmd — Query Markup Documents." GitHub.